SOFTWARE ENGINEERING

Complete Study Notes

Software Testing · SDLC · Development Models · Estimation · DFD · Object-Oriented Design · Quality & CMM · Maintenance

Typed from the handwritten class notes — every topic, example and diagram retained, with clarifications where the page was unclear or needs a correction.

Contents

1. Software Engineering — Introduction **3**

Characteristics . . . 3

2. Software Testing **3**

Alpha vs Beta testing . . . 4

Black-box vs white-box vs grey-box testing . . . 5

3. SDLC — Software Development Life Cycle **6**

Verification vs Validation . . . 6

SRS — Software Requirement Specification . . . 7

4. Software Development Models **8**

Comparison of the models (summary table added for revision) . . . 13

5. Estimation in a Software Project **14**

5.1 Size estimation . . . 14

5.2 Cost estimation . . . 14

5.3 Staffing-level estimation (work force) . . . 14

6. DFD — Data Flow Diagram **15**

DFD components . . . 15

Types (levels) of DFD . . . 15

Rules of DFD . . . 16

7. Object-Oriented Design **16**

Characteristics . . . 16

Principles of OO design — SOLID . . . 17

8. Software Reliability & Quality Assurance (SRQA) **18**

Quality characteristics . . . 18

Musa’s basic model . . . 18

ISO 9000 series . . . 18

CMM — Capability Maturity Model . . . 18

9. Software Maintenance **19**

Types of maintenance . . . 19

Software maintenance process models . . . 20

Reverse engineering . . . 20

Appendix — Clarifications and corrections **21**

1. Software Engineering — Introduction

- **Software engineering** is the kind of engineering in which software is developed for a computer system or any electronic device.
- Father of software engineering → **Barry Boehm** (as written in the notes).

Characteristics

#	Characteristic
1	Large software
2	Cost
3	Dynamic nature
4	Quality management
5	Scalability

2. Software Testing

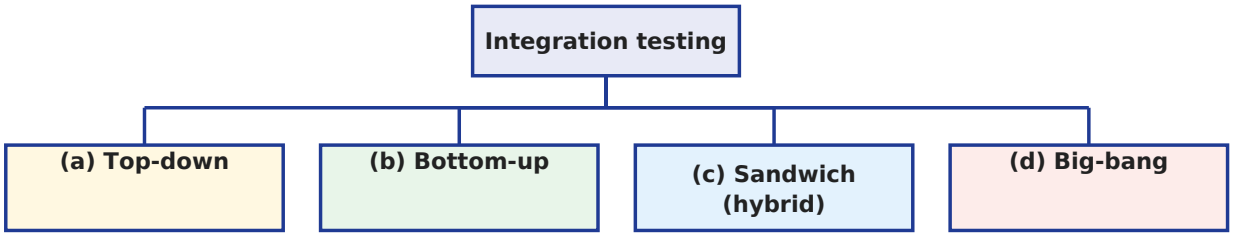
- **Software testing** is the process of executing software whose main objective is to **find bugs**.

(1) Unit testing

- Also known as **component testing / module testing / function testing**.
- Done during the **coding phase**.
- Done by the **programmer**.

(2) Integration testing

- Testing by **combining all units / components together**.
- To see all the units function together.
- 4 types:



Types of integration testing.

Type	Meaning
(a) Top-down (written “top-bottom”)	Units are combined starting from the top of the hierarchy
(b) Bottom-up	Units are combined starting from the bottom
(c) Sandwich	**Top-down + bottom-up** — a hybrid
(d) Big-bang	All units are combined in one go and tested together; **best for a small system**

(3) Alpha testing

- Done **before releasing** the software to users, to make it error and issue-free.
- Environment used: **developer environment**.
- Requires **coding knowledge**.
- Performed by the **software engineer**.

(4) Beta testing

- Testing done by **customers**.

(5) Stress testing

- Used to verify the **stability and reliability** of software.
- Checks how much load the software can bear and at what load it will crash.
- **Non-functional** testing.
- Also known as **endurance testing** or **torture testing**.

(6) Recovery testing

- Validates whether, when the software crashes, it can **recover** or not.
- Performed by the testing team.

(7) Security testing

- Checks whether the software can keep data **secure** or not.
- Checks how data security is maintained if the software is hacked.
- **Non-functional** testing.

(8) Smoke testing

- **Build verification testing**.
- When the development team provides a **new build (version)**, testing validates the build and checks whether it is error-free.

(9) Regression testing

- Ensures that changes made in the software are **not causing any new issues** in its working.
- It is done **repeatedly**.

(10) System testing

- Checks whether the developed software works on **as many operating systems** as required or not.
- A type of **black-box** testing.

(11) Performance testing

- Done by a **performance engineer**.
- Used to test **runtime performance**; checks software speed.

(12) Ad-hoc testing

- Performed **without any planning or documentation**.

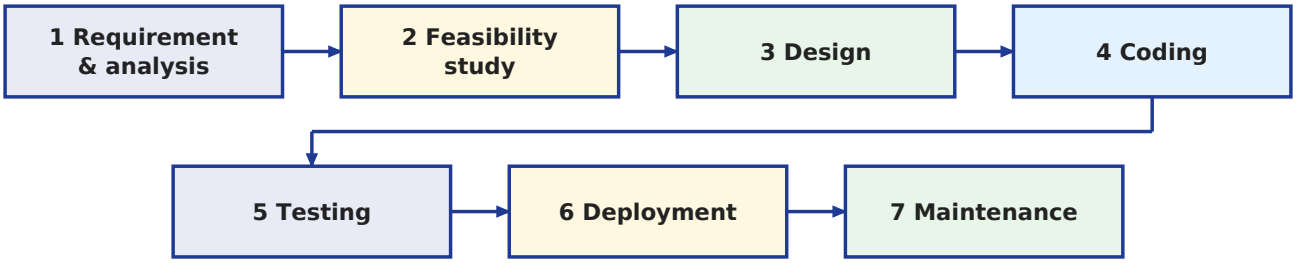
Alpha vs Beta testing

#	Aspect	Alpha	Beta
1	Objective	Identify internal errors & validate core functionalities	Gather real-world feedback & validate usability
2	Participants	Software developers & Quality Assurance team	External users (limited)
3	Focus	Functionality, performance, security	User experience, compatibility
4	Timing	Before beta	Before final release
5	Method	White & black box testing	Black box & exploratory testing
6	Risk	Low risk and controlled	High risk due to large audience

Black-box vs white-box vs grey-box testing

Aspect	Black box	White box	Grey box
Knowledge of code	No knowledge	Full knowledge	Partial knowledge
Focus	Functionality / behaviour	Internal code & logic	Combination of both
Performed by	Tester, Quality Assurance engineer	Developer	Testers with some knowledge
Testing approach	Input–output validation	Code-path analysis	Both functional & structural testing

3. SDLC — Software Development Life Cycle



The seven SDLC phases in order (drawn from the list in the notes).

(1) Requirement and analysis

- Details are gathered about **what we want the software to achieve**.

(2) Feasibility study

- A detailed study of the requirements is done and the **possibility of the software project's success** is checked.
- **Cost analysis** of the project is also done.
- The objective is **not to solve the problem**, but to decide whether the problem can be solved or not.

Type of feasibility	Question it answers
(a) Technical	Can it be built with the available technology?
(b) Operational	Will it work in the organisation / for the users?
(c) Economical	Is it worth the cost?

The one-line questions in the right column are added for clarity; the notes only list the three types.

(3) Design

- The **blueprint** of the system / software.
- The system is designed using the complete technical specification.

(4) Coding

- The coding work — the **real software is prepared** in this phase.

(5) Testing

- Code **verification and validation** are done in this phase.
- It is a **long-lasting** phase.

(6) Deployment

- Deployment of the completed software / system at the **end user**.

(7) Maintenance

- A **long-lasting** process.
- Types: **(a) Corrective (b) Adaptive (c) Perfective (d) Preventive** (see chapter 9).

Verification vs Validation

Verification	Validation
Performed by the **QA team** To check that the apps are according to the specification "Right product" (as written) Checks the product **White-box** testing	Executed by the **testing team** Focus on validating the actual product "Product is right" (as written) Checks functionality **Black-box** testing

Clarification

The usual wording is: **verification** = “are we building the product *right*?” (checking against the specification), **validation** = “are we building the *right* product?” (checking against what the user needs). The notes’ two phrases read the other way round, so treat the bullet points above as written and remember the standard definitions for exams.

SRS — Software Requirement Specification

- A **document / description of all requirements** needed to develop the software.
- Says **what features** it will have.
- Acts as a **contract between the client and the development team**.
- Reduces the time taken in the SDLC.
- The client can use the SRS to check whether the product **matches the requirements** or not.

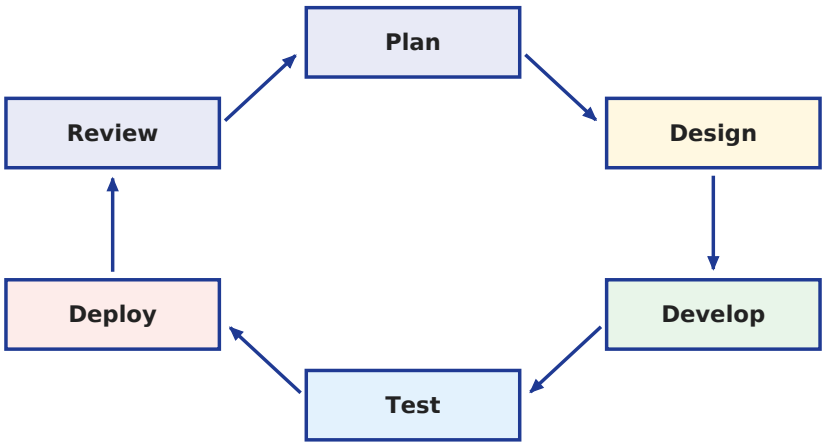
4. Software Development Models

(1) Agile

- A combination of **iterative and incremental** development.
- Main focus on **process adaptability and customer satisfaction**.
- All iterations / modules are kept **small**.

Principles

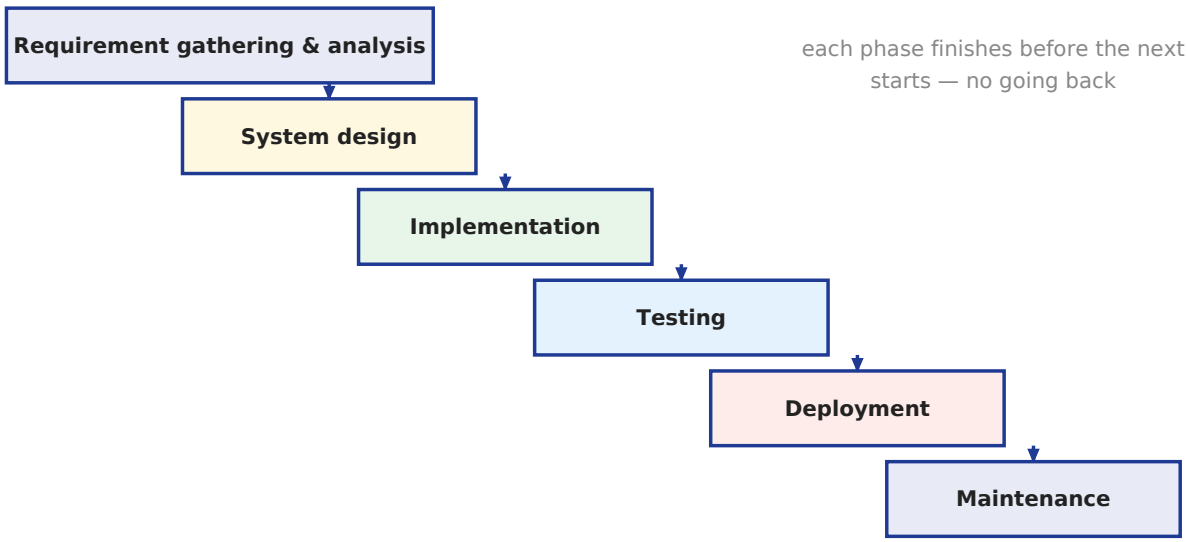
- A **customer representative** reviews and evaluates the requirements after every iteration.
- Does **not rely on documentation** — a demo of the working software is given.
- Usually **two programmers work together**; one writes code and the other reviews it (**pair programming**).



Agile cycle (as drawn in the notes: Plan → Design → Develop → Test → Deploy → Review → back to Plan).

(2) Waterfall model

- **Linear and sequential** model.
- A development phase does not start until the previous phase has finished.
- Phases **cannot overlap**.
- Once a phase is completed and you move to the next phase, you **cannot go back** to the previous phase.
- The **output of one phase becomes the input** of the next phase.

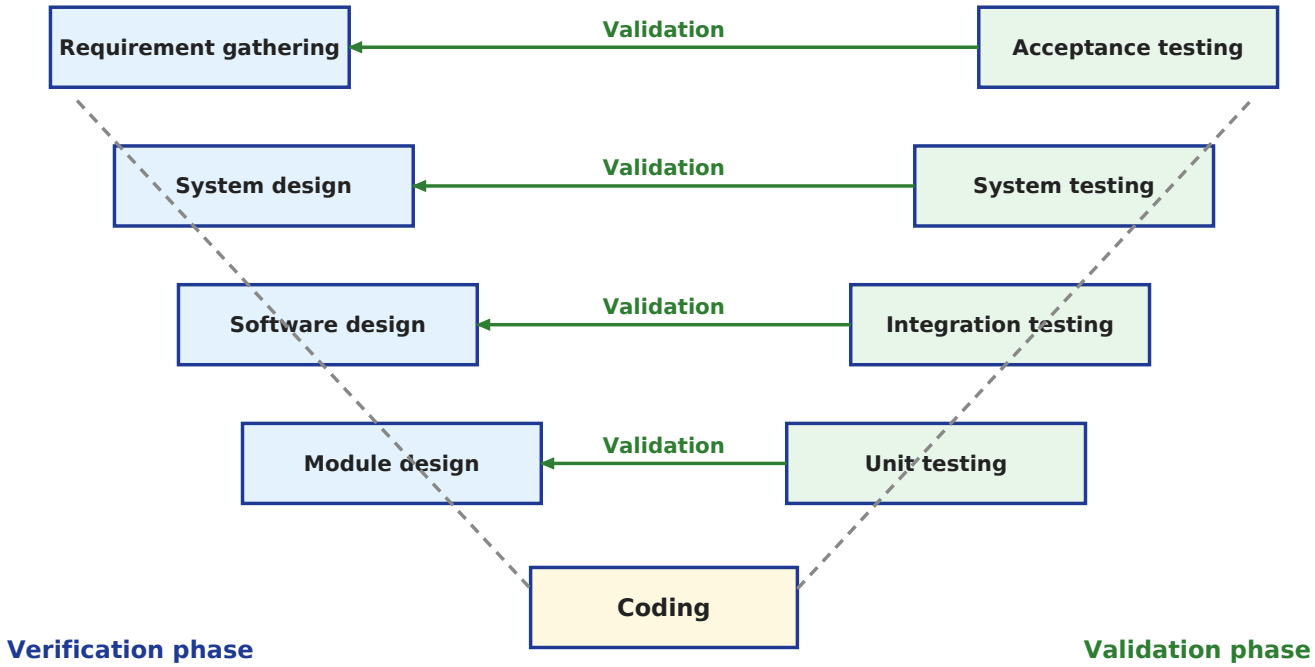


Waterfall model: requirement gathering → system design → implementation → testing → deployment → maintenance.

Advantages	Disadvantages
Timeline and deliverables for each step are fixed Mostly used for small projects with limited users	Lack of flexibility Less user involvement No risk management

(3) V-model

- **Verification and validation** model; an extension of the waterfall model.
- Shaped like a **“V”**: development and testing run in parallel.



V-model: each development phase on the left is validated by a testing phase on the right.

Development phase (verification side)	Matching test (validation side)
Requirement gathering	Acceptance testing

Development phase (verification side)	Matching test (validation side)
System design	System testing
Software design	Integration testing
Module design	Unit testing
Coding (bottom of the V)	—

Advantages	Disadvantages
Dedicated testing step for every development step Strong emphasis on documentation Because testing is done in every phase, quality and reliability are better	Not flexible Not useful for large and complex projects

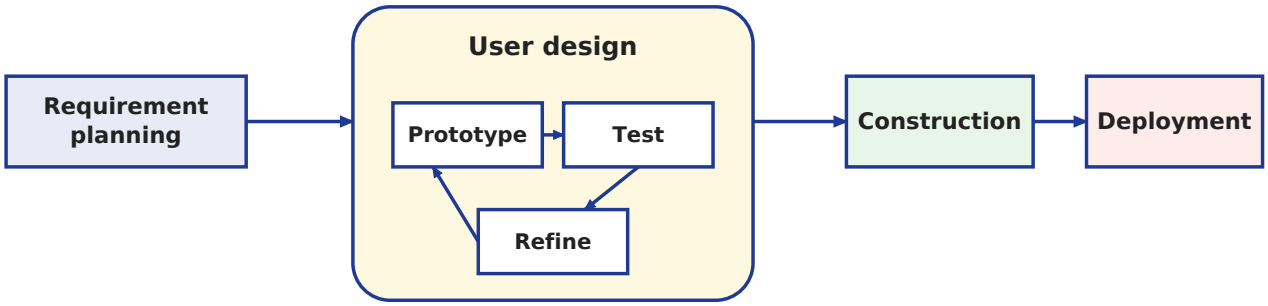
(4) Incremental model

- The whole system is **not built at once**; it is developed in separate **increments**.
- In each increment the new part is added to the already developed software.

Advantages	Disadvantages
Easy testing, less risk Easy to change according to the client's needs	Requires very good design and planning Complexities can arise

(5) RAD model — Rapid Application Development

- Understand the user's requirements, and **quickly** develop the software stably.
- Based on **prototypes**.
- Phases: **(i) Requirement planning (ii) User design (iii) Construction (iv) Deployment**.

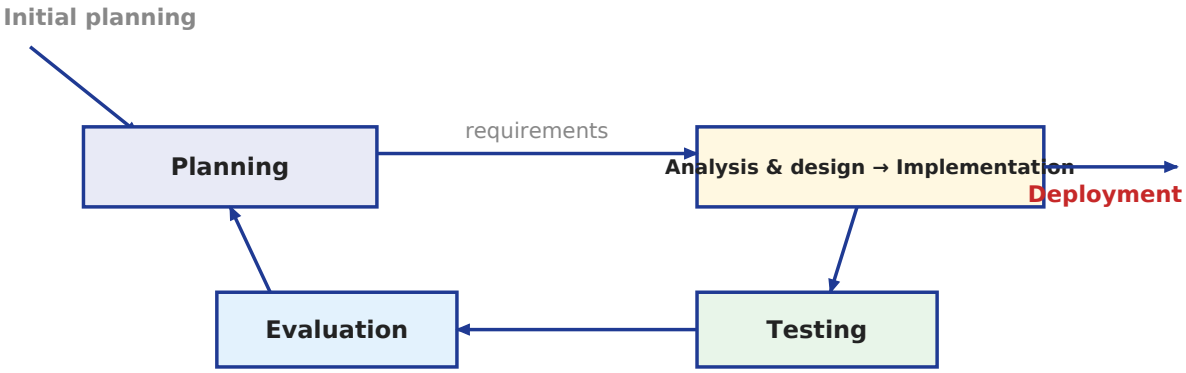


RAD: requirement planning → user design (prototype → test → refine loop) → construction → deployment.

Advantages	Disadvantages
Development takes less time High-quality software is built quickly Repeated feedback makes the software better	Difficult for less-experienced developers Useful only for small and medium-size projects

(6) Iterative model

- The software is developed in **small parts**.
- Each step includes **testing and review**.
- The software is not built all at once.
- **Characteristics:** phased development; customer feedback; reduced risk.
- **Phases:** (i) Requirement gathering (ii) Design (iii) Implementation (iv) Testing (v) Evaluation.



Iterative model: initial planning, then planning → implementation → testing → evaluation → planning, until deployment.

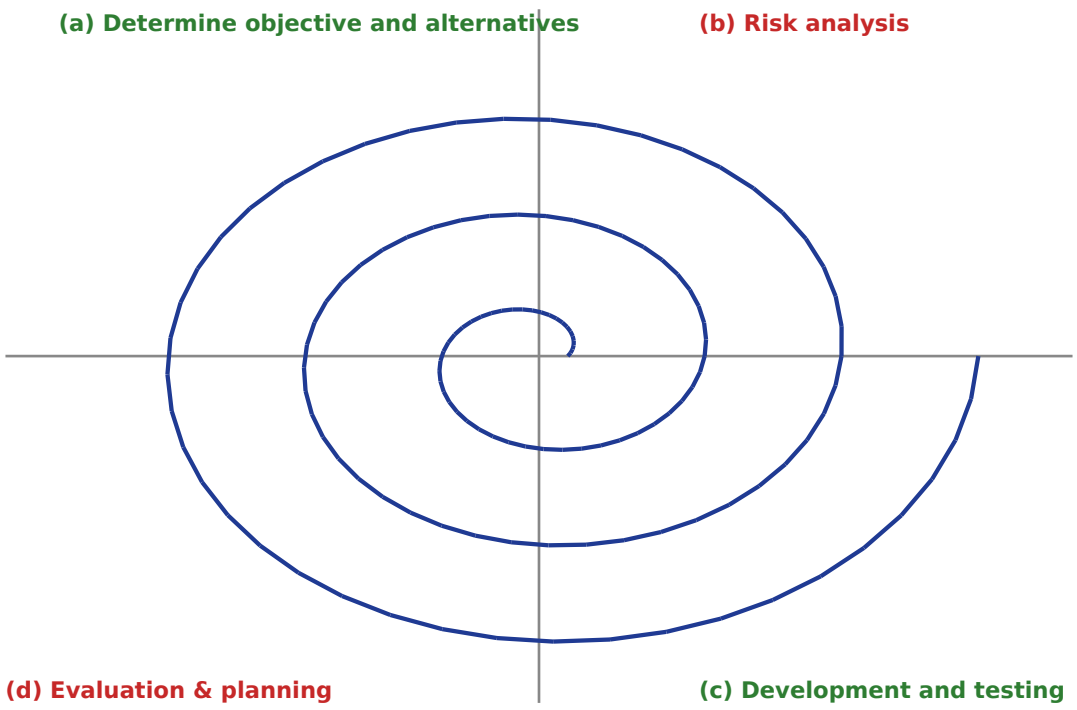
Advantages	Disadvantages
Working software is obtained quickly Easy to develop large software in small parts	Time and cost increase because of many iterations

(7) Spiral model

- **Reduced project risks**.
- Called spiral because development proceeds in irregular iterations (cycles).

Four major activities
(a) Determine objective and alternatives
(b) Risk analysis
(c) Development and testing
(d) Evaluation & planning

- **Cycle of the spiral:** the four activities are repeated; after each cycle the software becomes **more mature**; a combination of the waterfall and iterative models.
- Features: risk management; interactive and iterative development; system suitability.

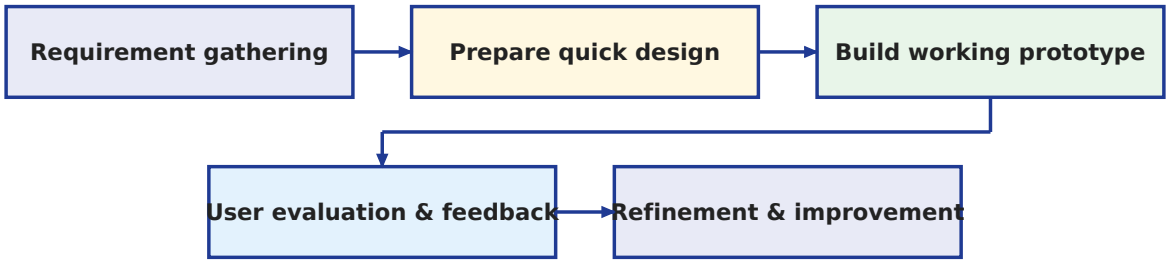


Spiral model: the four activities repeat in every loop of the spiral (illustration added; the notes list the activities only).

Advantages	Disadvantages
Better risk management Regular customer feedback Flexibility	More time & cost Not suitable for small projects

(8) Prototype model

- Used when the **customer's requirements are not clearly defined**.
- A **working model** is built so that the customer can see how it works and suggest necessary changes.



Prototype model process steps (flow drawn from the list in the notes).

Advantages	Disadvantages
Requirements can be understood clearly	Not suitable for large and complex projects

(9) COCOMO model — Constructive Cost Model

Type	Details
(a) Basic COCOMO	Simplest; depends **only on software size** (KLOC = thousand lines of code). 3 categories: (i) **Organic** — small and easy; (ii) **Semi-detached** — moderate; (iii) **Embedded** — complex & hard product
(b) Intermediate COCOMO	—
(c) Detailed COCOMO	—

(10) Big-bang model

- Coding starts **without proper planning** and without fully understanding the customer's requirements.
- Developed and delivered once; **no testing**.

Advantage	Disadvantages
Suitable for small projects	High risk No documentation

(11) Kanban model

- Started by the **Toyota** company.
- Used to **manage work flow**.

Comparison of the models (summary table added for revision)

Model	Key idea	Best / used for	Main drawback
Agile	Iterative + incremental; small iterations; pair programming	Changing requirements; customer involvement	—
Waterfall	Linear, no going back	Small projects, limited users	No flexibility / risk management
V-model	Test phase for every development phase	Quality-critical work	Not flexible; not for large complex projects
Incremental	Built in separate increments	Easy testing, less risk	Needs very good design
RAD	Prototype based, fast	Small / medium projects	Hard for less-experienced developers
Iterative	Small parts, test + review each step	Large software in small parts	Time and cost rise
Spiral	Four activities per cycle	Risk management	More time & cost; not for small projects
Prototype	Working model first	Unclear requirements	Not for large complex projects
Big-bang	Code with no planning	Small projects	High risk, no documentation

5. Estimation in a Software Project

5.1 Size estimation

(A) Size measurement

#	Measure	Meaning
1	**LOC** — Line of Code	Counts the total number of lines written
2	**FPA** — Function Point Analysis	Measures the task the user expects from the software
3	**UCP** — Use Case Points	Based on the use-case diagram

(B) Size estimation techniques

#	Technique	How it works
1	**Expert judgement**	Estimation based on the experience of experts
2	**Delphi technique**	Take estimates from several different experts and calculate their **average**
3	**Analogy-based**	Estimate using the last completed similar project as a reference
4	**Top-down**	First estimate the whole system, then break it into smaller parts
5	**Bottom-up**	Opposite of top-down — estimate the small parts first and add them up

5.2 Cost estimation

Types of cost estimate

Type	Used for
(A) ROM — Rough Order of Magnitude	The **initial estimate**
(B) Budget estimate	Used for **planning**
(C) Definitive estimate	Made when **complete information** about the project is available

Factors affecting cost

- Size of the project
- Complexity of the project
- Team experience
- Technology used
- Onsite & offshore
- Deadline

Cost estimation techniques

#	Technique	Details
1	**Algorithmic / parametric methods**	(a) COCOMO model (b) FPA
2	**Empirical method**	(a) Expert judgement (b) Delphi technique
3	**Analogous estimate**	Use data from previously completed similar projects
4	**Top-down**	First the whole project, then the smaller parts
5	**Bottom-up approach**	Opposite of top-down

5.3 Staffing-level estimation (work force)

- **(A) Create the WBS** (Work Breakdown Structure): divide the project into small parts; fix the required time and staff for each part.

- **(B) Effort estimation**: find how many hours are needed to complete each task.

6. DFD — Data Flow Diagram

- A **graphical tool** that shows the system as a **single process** (and, at lower levels, its sub-processes).
- Known as a **bubble chart**.

DFD components



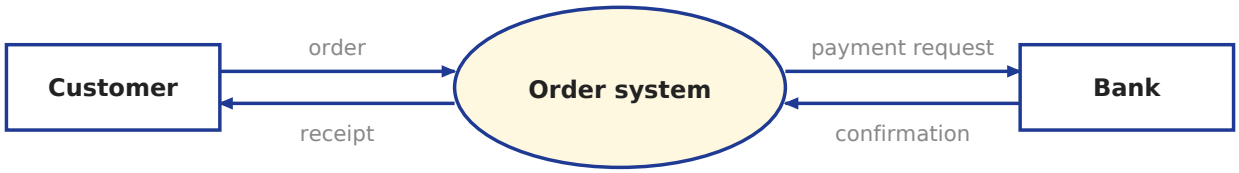
The four DFD symbols.

#	Component	Meaning	Symbol	Example
1	**External entity**	People and organizations that interact with the system; they send data to the system or receive data from it	Rectangle	Bank, customer etc.
2	**Process**	The work of processing data in the system	Circle / ellipse	Processing an order, validating data
3	**Data store**	A place where data is stored temporarily or permanently — file, database	Open-ended rectangle (=)	Orders file
4	**Data flow**	Shows how, and from where to where, data moves inside the system; shows the direction of data	Arrow	—

Types (levels) of DFD

(1) Level 0 / context-level DFD

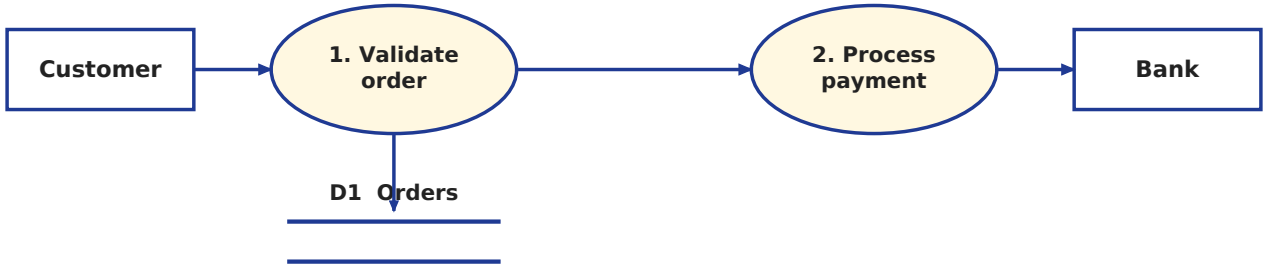
- The **highest-level view** of the system.
- The whole system is shown as a **single process**.
- Only the **external entities and the main data flows** are shown.



Example (added): context-level DFD of an order system — one process, two external entities.

(2) Level 1 DFD

- The main process is **divided into sub-processes**.
- Shows how data flows between these sub-processes.



Example (added): level-1 DFD splitting the order system into two sub-processes and a data store.

(3) Level 2, 3 (detailed DFD)

- Level 1 explained in **more detail**.
- Used to show the system in **greater depth**.

Rules of DFD

- Every process must have **at least one input and one output**.
- Data **cannot go directly from one data store to another** data store.
- There is **no data flow directly between external entities**.
- One process should not communicate directly with another process (as written in the notes).
- While data is being processed, its flow is **not shown in two opposite directions**.

Clarification

Many textbooks do allow a data flow from one process straight to the next process (as in the level-1 example above, where “Validate order” feeds “Process payment”). Follow the notes’ wording in your course exam, but know that this rule is stated differently elsewhere.

7. Object-Oriented Design

Characteristics

#	Concept	Meaning (from the notes)
1	**Class**	Template used to create objects
2	**Object**	An instance of a class
3	**Encapsulation**	Keeps data secure from external access
4	**Inheritance**	Uses the properties of one class in another class
5	**Abstraction**	Hides complex details, shows the required ones
6	**Polymorphism**	Using a function with the same name in different ways

Principles of OO design — SOLID

Letter	Principle	Rule
S	SRP — Single Responsibility Principle	Every class should have **one responsibility / one reason to change**
O	Open / Closed Principle	Software entities should be **open for extension but closed for modification**
L	Liskov Substitution Principle	A **subclass should be usable in place of its superclass**
I	Interface Segregation Principle	No client should be forced to depend on interfaces it does not use
D	Dependency Inversion Principle	Clients should depend only on **abstractions**, not on concrete implementations

8. Software Reliability & Quality Assurance (SRQA)

- **Software quality** = whether the software fulfils the user requirements or not.

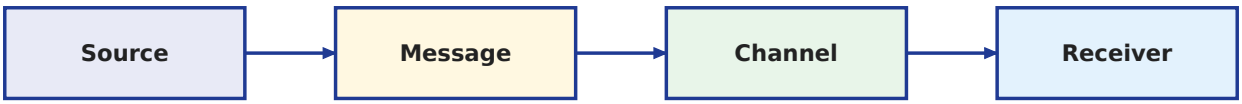
Quality characteristics

(a) Functionality	(b) Reliability	(c) Usability	(d) Efficiency
(e) Portability	(f) Maintainability	(g) Correctness	

Term	Meaning
Quality Assurance (QA)	A **process** that ensures whether a software product is following the **predefined standard** or not
MTBF — Mean Time Between Failure	The **average time** for which the software keeps working without failing
Quality Control (QC)	A **technical process** that tests the software to determine its quality
Standards	**ISO 9001**; **CMMI** (Capability Maturity Model Integration) for process improvement; **Six Sigma** for reducing errors / risks

Musa's basic model

- As written in the notes: a model of **instructional technology**, created by Musa.
- Used in the field of **education** to make effective communication and the teaching–learning process better.
- Four components:



The four components of the model as listed in the notes.

- **Purpose:** to make communication effective; to make techniques easy and simple; to help teachers understand how information reaches students.

> **Clarification**
> In software engineering, "Musa's basic (execution-time) model" normally means a **software reliability growth model** by John Musa that predicts failures and reliability during testing. The handwritten page describes a communication / instructional model instead — worth checking with your teacher which one your syllabus expects.

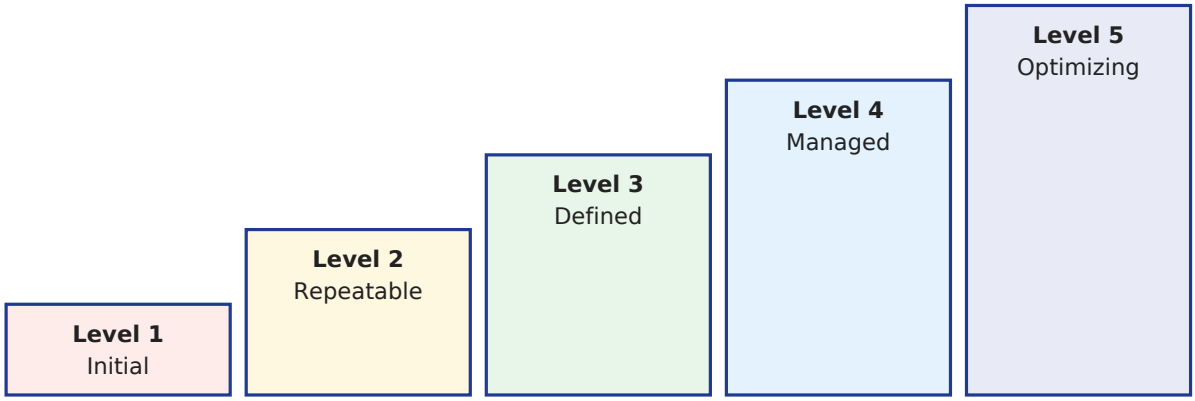
ISO 9000 series

- **International Organization for Standardization.**
- ISO 9000 = **QMS** (Quality Management System).
- Main characteristics:

(a) Customer focus	(b) Leadership	(c) Engagement of people	(d) Process approach
(e) Continuous improvement	(f) Relationship management	(g) Evidence-based decision making	

CMM — Capability Maturity Model

- **SEI-CMM** (Software Engineering Institute Capability Maturity Model).
- Used for **process maturity** in a software organisation.



The five CMM levels — maturity increases from left to right.

Level	Name	Meaning
1	Initial	No fixed process; no fixed manners
2	Repeatable	Repeat the successful process
3	Defined	Process documentation is defined
4	Managed	Process & quality management
5	Optimizing	Regular improvement

Feature	ISO 9000	SEI CMM
1) Type	QMS (Quality Management System)	Process maturity model
2) Focus	Quality assurance & customer satisfaction	Software process improvement
3) Certificate	By ISO	No certification
4) Process approach	Requirement of process	Maturity of process

9. Software Maintenance

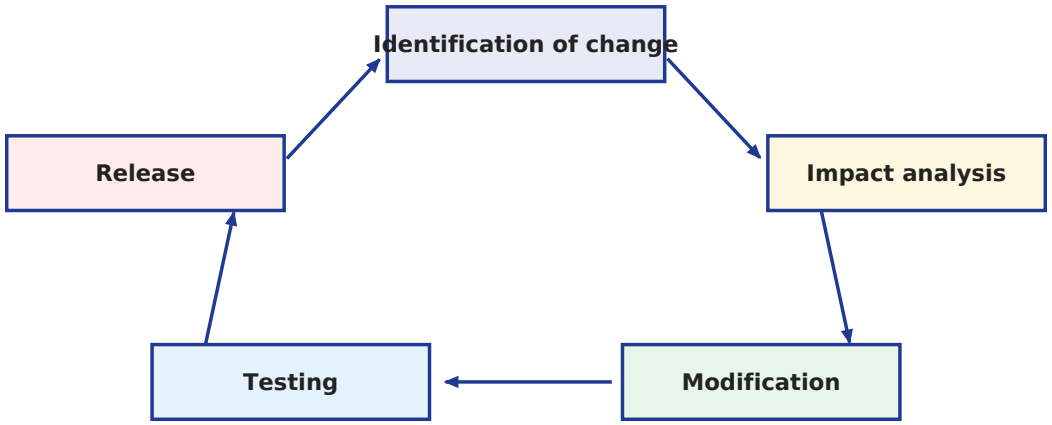
- **Timely correction and modification** of software so that it keeps working smoothly.

Types of maintenance

Type	Purpose	Example / detail
(A) Corrective	Correct bugs and errors	Runtime and logical errors
(B) Adaptive	Make the software capable of running on **new hardware or OS**	—
(C) Perfective	Make a **new, better version** of the old software available	—
(D) Preventive	Updates made available for **security against future problems and errors**	—

Software maintenance process models

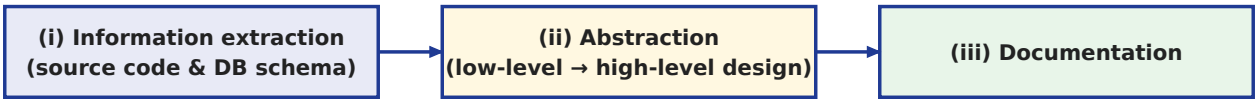
Model	Idea
(a) Quick-fix model	Concentrated on **directly solving the problem**
(b) Iterative enhancement model	Slowly, step by step, making corrections in the current system
(c) Reuse-orientation model	Reusing available code and content
(d) Boehm's model / SMW (Software Maintenance Wheel)	A **circular** model that includes identification of change, impact analysis, modification, testing and release (written "rebase")



Boehm's Software Maintenance Wheel: identification of change → impact analysis → modification → testing → release.

Reverse engineering

- A process in which we **fetch the design and documentation** of a currently available software **without changing its code**.
- 3 steps:



The three steps of reverse engineering.

Step	What happens
(i) Information extraction	Source code and DB schema are examined
(ii) Abstraction	Converting from low-level to high-level design
(iii) Documentation	The recovered design is documented

Appendix — Clarifications and corrections

The handwritten pages were typed as faithfully as possible. These places were clarified, flagged or marked as added:

Topic	In the handwritten notes	Here
Barry Boehm	“Father” of software engineering	Kept as written (he is best known for the spiral model and COCOMO)
Integration testing	“Top-bottom”	Top-down
Verification vs validation	“Right product” (verification) and “Product is right” (validation)	Kept; standard wording is the reverse — flagged
DFD rule	A process should not communicate directly with another process	Kept; flagged — many texts allow process-to-process flows
Musa’s basic model	Described as an instructional / communication model	Kept; flagged — in software engineering it is normally a reliability model
Maintenance wheel	“rebase”	Read as “release”
Feasibility	Types: technical, operational, economical	One-line questions added for clarity
Added for revision	—	Model comparison table, SDLC / prototype / reverse-engineering flow pictures, spiral picture, DFD examples (level 0 and level 1)